wsdm-cup-multilingual-chatbot-arena

# 7th Place Solution

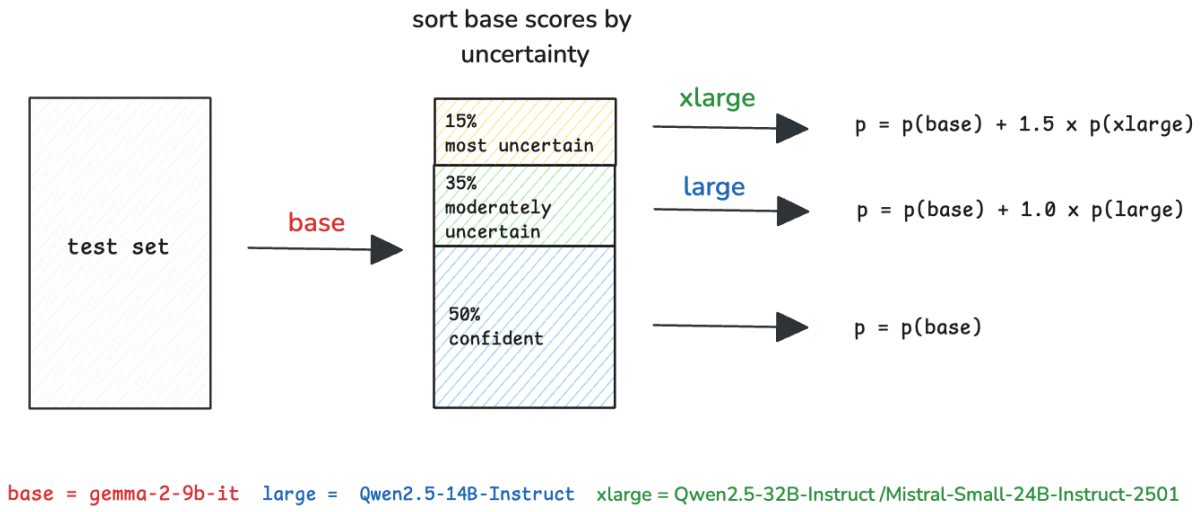
| | |
|---|---|
| **Rank** | 7 |
| **Team** | Team Turing |
| **Author** | Raja Biswas |
| **Topic ID** | 567589 |
| **Version** | Original |

Source: https://www.kaggle.com/competitions/wsdm-cup-multilingual-chatbot-arena/discussion/567589
Retrieved with Kaggle CLI on 2026-10-21.

Thanks to Kaggle, the competition hosts, and the participants for making this an exciting competition! We are pleased to see the winning solutions generalize well to the test set. Congratulations to all the winners! Trushant and I had a lot of fun building our solution over a 3 week sprint.

## Inference Pipeline

We implemented an uncertainty-based ensemble pipeline as shown in the figure below:




- First, a fine-tuned `gemma-2-9b-it` model was used to score all test examples
- Next, uncertainty scores were calculated as closeness to the decision boundary (`uncertainty = 1-|p-0.5|`)
- Top 50% uncertain examples were identified (i.e. those with probabilities closest to 0.5) and scored again using with a larger model.

Specifically,

- For top 15% uncertain examples, we used a fine-tuned `Qwen2.5-32B-Instruct` (selected submission 1) or `Mistral-Small-24B-Instruct-2501` (selected submission 2) model (xlarge)
- For next 35% examples, we used `Qwen2.5-14B-Instruct` (large)
- For remaining examples, we used scores directly from the `gemma-2-9b-it` model (base)

The final score was computed as an ensemble:

- Top 15% uncertain examples: `Score = 1.0 x base + 1.5 x xlarge`
- Next 35% examples: `Score = 1.0 x base + 1.0 x large`
- Remaining examples: `Score = 1.0 x base`

We used vllm for inference. The inference time estimates for 512 examples are shown below together with max sequence length:

```
||||
||||
||||
```

```
||||
||||
||||
```

# Data

We used the following data sources for training:

## Labelled Datasets

- WSDM Cup - Multilingual Chatbot Arena Competition Data (45k)
- LMSYS - Chatbot Arena Human Preference Predictions Competition Data (30k)
- lmarena-ai/PPE-Human-Preference-V1 (10k)

## Unlabelled Datasets

- wsdm - open models - nbroad (Dataset from Nicholas - Sampled 6k)
- Skywork/Skywork-Reward-Preference-80K-v0.2 (Sampled 25k)
- allenai/tulu-3-sft-olmo-2-mixture (Sampled 30k)
- Team generated preference pairs (100k examples)
  - Prompts from allenai/WildChat-1M and lmsys/lmsys-chat-1m datasets
  - Cluster based sampling to roughly match the distribution from Arena Explorer

# Models

We fine-tuned models from Gemma, Llama, Qwen and Mistral families, which achieved the following scores:

| Model | Dataset | # | Validation Accuracy | Validation AUC | Public LB |
|---|---|---|---|---|---|
| gemma-2-2b-it | Comp + 10k Labelled + 80k Pseudo Labelled | | 0.676 | 0.742 | 0.675 |
| gemma-2-9b-it | Comp + 30k Labelled + 120k Pseudo Labelled | | 0.702 | 0.777 | 0.696 |
| Qwen2.5-14B-Instruct | Comp + 30k Labelled + 120k Pseudo Labelled | | 0.699 | 0.777 | 0.696 |
| Qwen2.5-32B-Instruct | Comp + 30k Labelled + 120k Pseudo Labelled | | 0.701 | 0.780 | x |
| Mistral-Small-24B-Instruct-2501 | Comp + 30k Labelled + 120k Pseudo Labelled | | **0.704** | **0.783** | x |
| Qwen2.5-72B-Instruct | Comp | | 0.704 | 0.779 | x |
| Llama-3.1-Nemotron-70B-Instruct | Comp | | 0.693 | 0.770 | x |

- We first fine-tuned `Qwen2.5-72B-Instruct` and `Llama-3.1-Nemotron-70B-Instruct` on the competition subset and used them to generate pseudo labels for the unlabeled datasets.
- For Gemma models, we directly used "A" and "B" token logits to compute the preference probability. These models did not have a reward model head. They were trained with a combination of language modeling and reward modeling losses. (Reference: Regularizing Hidden States Enables Learning Generalizable Reward Model for LLMs)
- The remaining models were trained using the standard `AutoModelForSequenceClassification` architecture with a linear head.

## Generalization

We considered the following aspects in an attempt to improve generalization:

- Incorporated larger LLMs in the inference pipeline to handle uncertain examples. Larger LLMs tend to generalize better.
- Ensemble of different modelling approaches and model families (logits-based and AutoModelForSequenceClassification).
- Kept the preference modelling head simple (one linear layer).
- Distillation from 70B+ LLMs using pseudo labels on a large unlabeled dataset containing diverse prompts.

## Links

- Inference Notebook

We look forward to reading and learning from your solutions! Special thanks to @trushk for the perfect collaboration and teamwork!

## Supplemental Q&A

> **Cody_Null · 2025-03-11T03:16:08.237000**
>
> Wow great work! We wanted to do something like this but sadly couldn't get vllm to work with Gemma and we were unable to get great results from the qwen models initially and it seems we moved on too soon. I would love to hear more about how you got these models to work with vllm?

> **Raja Biswas · 2025-03-11T03:38:01.420000**
>
> Hi @cody11null, Thanks!
>
> For Gemma, we needed to disable logit soft-capping for both training and inference (else we had troubles in getting soft prob scores). Yes, Qwen models blended really well with Gemma. Please refer to our inference notebook for vllm inference.

> **ducnh279 · 2025-03-19T13:27:30.090000**
>
> Congrats @conjuring92 and your team!
>
> I read the reference paper and noticed that they **kept the language modeling head (`lm_head`) and added a separate reward head** for training with both language modeling and reward modeling losses. However, your approach **doesn't introduce a new reward head** - instead, it relies on the `lm_head` and uses the logits of specific pooling tokens ('A', 'B').
>
> Could you share the reasoning behind this choice? Have you tested both approaches, and did you find using `lm_head` to be more efficient while maintaining comparable performance?
>
> From my experience, when fine-tuning decoder models the `lm_head`, it usually requires a **smaller learning rate than the backbone** because it has many parameters and is already pre-trained. This contrasts with encoder models, where the classification head is often trained with a **larger** learning rate

than the backbone. In my case, I often **replace `lm_head` with a new head** that has a similar size as the embedding dim. I find that this significantly reduces parameter size and allows me to use a more standard learning rate, similar to fine-tuning encoder models.

By the way, your approach is an interesting and valuable technique for training reward models to enhance generalization. It reminds me of the method in paper this paper , "Fine-Tuning Can Distort Pretrained Features and Underperform Out-of-Distribution." In particular, Linear Probing (LP), which trains only the final linear layer, and LP-FT, which fine-tunes the entire model after LP, have proven effective for generalization.

**Raja Biswas · 2025-03-20T16:22:57.467000**

Hi @ducnh279 Thanks for the insightful comments & sharing your experience!

> From my experience, when fine-tuning decoder models the lm_head, it usually requires a smaller learning rate than the backbone because it has many parameters and is already pre-trained. This contrasts with encoder models, where the classification head is often trained with a larger learning rate than the backbone.

I have the same experience. I typically use 10x lower lr for the `lm_head` as compared to other param groups.

> I read the reference paper and noticed that they kept the language modeling head (lm_head) and added a separate reward head for training with both language modeling and reward modeling losses. However, your approach doesn't introduce a new reward head - instead, it relies on the lm_head and uses the logits of specific pooling tokens ('A', 'B').

Yes, good catch. We kept single head to simplify the implementation and assuming it should have even stronger regularization effect. We didn't get to benchmark the performance vs the original approach proposed in the paper.

> By the way, your approach is an interesting and valuable technique for training reward models to enhance generalization. It reminds me of the method in paper this paper , "Fine-Tuning Can Distort Pretrained Features and Underperform Out-of-Distribution." In particular, Linear Probing (LP), which trains only the final linear layer, and LP-FT, which fine-tunes the entire model after LP, have proven effective for generalization.

Nice paper! Thanks for mentioning this. If we expect a large distribution shift, it might be better to just fine-tune the final layer. Otherwise, we might be under-training the model.